

Projeto MICROC – Etapa 4

Análise Semântica 1: nomes, escopos e tipos

2º semestre de 2026

Objetivo

Nesta etapa, o grupo implementará as duas primeiras passagens da análise semântica da MICROC. O analisador recebe o objeto `Program` construído pelo parser, resolve declarações e usos, determina tipos estáticos e rejeita os programas que violam as regras desta entrega.

Resultado da etapa

objeto `Program` → resolução de nomes → verificação de tipos → AST anotada
ou diagnósticos

O analisador percorre a AST existente. Ele não deve reler tokens, reconhecer sintaxe nem substituir os nós por outra representação.

1 Material inicial e entrega

O starter contém os contratos do lexer, do parser e da AST publicados nas etapas anteriores, além dos novos módulos:

- `symbols.py`, com as estruturas fundamentais de símbolos e escopos;
- `semantic_errors.py`, com categorias e diagnósticos;
- `name_resolver.py` e `type_checker.py`, com as passagens a completar;
- `semantic.py`, que coordena as passagens;
- `runner.py` e `test.mc`, para executar o pipeline completo; e
- testes públicos, configuração do `pytest` e GitHub Actions.

Substitua `Lexer.py` e `parser.py` pelas implementações entregues pelo próprio grupo. O novo starter não substitui automaticamente o trabalho das etapas anteriores. Não altere os nomes ou campos públicos dos tokens e da AST. Não altere `symbols.py` nem `semantic_errors.py`. Esses contratos serão reutilizados nas próximas etapas do compilador. É permitido criar visitantes, classes, tabelas e funções auxiliares em outros módulos. As interfaces e metadados publicados neste documento devem ser preservados.

Prazo desta etapa

A entrega encerra-se no **domingo, 4 de outubro de 2026, às 23h59**, no horário de Brasília. O commit considerado na correção deve estar no repositório correto do grupo no GitHub até esse horário.

2 Interface pública

A entrada da etapa é uma AST sintaticamente válida:

```
program = Parser(Lexer(source).scan()).parse()
program = SemanticAnalyzer().analyze(program)
```

`analyze` deve executar as duas funções acima na ordem apresentada. Em caso de sucesso, devolve o mesmo objeto `Program`, agora anotado. Uma passagem sem erros retorna `None`; uma passagem com erros lança `SemanticError`. Erros de resolução impedem a passagem de tipos, pois esta depende dos vínculos produzidos pela primeira.

3 Símbolos, escopos e metadados

3.1 Estruturas fornecidas

`SymbolKind` diferencia função, parâmetro e variável. Todo `Symbol` expõe `name`, `kind`, `type` e `declaration`. Para função, `type` é o retorno. O campo a seguir contém os tipos dos parâmetros na ordem declarada:

```
FunctionSymbol.parameter_types
```

Cada `Scope` possui `parent` e um dicionário local `symbols`. Os campos, enums e classes desse arquivo formam um contrato fechado. O grupo decide onde implementar

as operações de declaração e procura e pode criar outras estruturas auxiliares. Objetos de símbolo representam identidade: dois `x` de escopos diferentes não são o mesmo símbolo.

3.2 Anotações obrigatórias

Use exatamente estas chaves de metadata:

Chave	Aplicação
<code>symbol</code>	símbolo resolvido em <code>FunctionDecl</code> , <code>Parameter</code> , <code>VarDecl</code> , <code>IdentifierExpr</code> e <code>CallExpr</code>
<code>scope</code>	escopo representado por cada <code>Block</code>
<code>type</code>	<code>TypeName</code> determinado para cada <code>Expr</code>

Declaração e seus usos devem apontar para o mesmo objeto de símbolo. Cada bloco recebe um objeto de escopo distinto. Em programa válido, toda expressão recebe `type`; uma chamada `void` válida como comando recebe `TypeName.VOID` apenas como anotação interna.

4 Passagem 1: resolução de nomes

Todas as assinaturas de função devem ser coletadas antes da análise dos corpos. Isso permite chamadas antecipadas, recursão direta e indireta. Nomes de funções são únicos e ocupam espaço de nomes separado de parâmetros e variáveis.

Para cada função, o bloco externo cria um escopo. Seus parâmetros são declarados nesse mesmo escopo; portanto, não podem ser redeclarados diretamente no corpo. Cada bloco interno cria um filho e pode ocultar nomes externos.

Comandos e expressões devem ser visitados em ordem de fonte. Uma variável entra no escopo antes da visita de seu inicializador. Assim, em `int x = x;`, o uso refere-se ao novo `x`; a futura análise de inicialização decidirá que essa leitura é inválida.

`IdentifierExpr` procura somente em escopos de variáveis. `CallExpr` procura somente na tabela global de funções. A procura em escopos começa no mais interno e segue pelos pais. São erros desta passagem:

- função repetida;
- parâmetro ou variável repetido no mesmo escopo;
- variável usada sem declaração visível;
- chamada de função inexistente; e
- ausência de `main` ou assinatura diferente de `int main()`.

Ao encontrar uma função repetida, preserve a primeira entrada da tabela global. Continue percorrendo todos os corpos e filhos para acumular erros independentes.

5 Passagem 2: verificação de tipos

5.1 Declarações e literais

Funções podem retornar `int`, `bool` ou `void`. Parâmetros e variáveis aceitam somente `int` ou `bool`. Um literal inteiro deve estar entre 0 e $2^{63} - 1$, inclusive. Como o sinal é um operador separado, `-9223372036854775808` contém um literal fora desse intervalo e deve ser rejeitado nesta etapa.

5.2 Expressões e operadores

Construção	Tipos exigidos	Tipo produzido
literal inteiro	—	<code>int</code>
literal booleano	—	<code>bool</code>
identificador	tipo do símbolo	mesmo tipo
- unário	<code>int</code>	<code>int</code>
!	<code>bool</code>	<code>bool</code>
+ - * / %	<code>int</code> , <code>int</code>	<code>int</code>
< <= > >=	<code>int</code> , <code>int</code>	<code>bool</code>
== !=	dois <code>int</code> ou dois <code>bool</code>	<code>bool</code>
&&	<code>bool</code> , <code>bool</code>	<code>bool</code>
chamada	assinatura declarada	retorno da função

Não existem coerções implícitas. Mesmo quando uma subexpressão falhar, visite os demais filhos para encontrar erros independentes. Use internamente um estado de tipo desconhecido para não produzir novos diagnósticos causados apenas por um erro anterior; esse estado não pertence a `TypeName` nem deve ser gravado nos metadados.

5.3 Contextos

- Inicializador e atribuição exigem exatamente o tipo da variável.
- Condições de `if` e `while` exigem `bool`.
- Chamadas exigem existência, aridade e tipo exato de cada argumento.
- Todos os argumentos são visitados, mesmo quando a aridade está errada.
- Chamadas `int` e `bool` podem ser comandos; o valor é descartado.
- Chamada `void` pode ser comando, mas não pode ser usada como valor.
- `return`; é aceito somente em função `void`.
- `return expressão`; exige o retorno não-`void` da função.
- `print` aceita strings, expressões `int` e expressões `bool`.

Esta etapa valida cada comando `return` encontrado, mas ainda não decide se todos os caminhos de uma função não-`void` retornam.

6 Diagnósticos

Cada `SemanticDiagnostic` possui `kind`, `message`, `span`, `line` e `column`. O atributo `diagnostics` de `SemanticError` é uma tupla não vazia. A correção considera

os diagnósticos produzidos, não uma ordem específica entre erros independentes. A redação de message pode variar, mas categoria e coordenada fazem parte da interface avaliada. A estrutura e a formatação fornecidas em `semantic_errors.py` não devem ser alteradas.

As categorias publicadas são:

<code>DUPLICATE_FUNCTION</code>	<code>DUPLICATE_DECLARATION</code>
<code>UNDECLARED_VARIABLE</code>	<code>UNDECLARED_FUNCTION</code>
<code>INVALID_MAIN</code>	<code>VOID_PARAMETER</code>
<code>VOID_VARIABLE</code>	<code>INTEGER_LITERAL_OUT_OF_RANGE</code>
<code>INVALID_UNARY_OPERAND</code>	<code>INVALID_BINARY_OPERANDS</code>
<code>INITIALIZER_TYPE_MISMATCH</code>	<code>ASSIGNMENT_TYPE_MISMATCH</code>
<code>CONDITION_TYPE_MISMATCH</code>	<code>ARITY_MISMATCH</code>
<code>ARGUMENT_TYPE_MISMATCH</code>	<code>VOID_VALUE_USED</code>
<code>RETURN_MISMATCH</code>	

Use o span da expressão ou uso que causou o erro. Quando a AST não separa o span do nome declarado, use o início do nó de declaração. Para `main` ausente, use o início de `Program`.

O runner escreve os diagnósticos em `stderr` e termina com status 1. Erros de uso ou leitura terminam com status 2. Durante o desenvolvimento, um erro de método ainda não implementado no scaffold pode terminar com status 3.

7 Limite para a Análise Semântica 2

Nesta entrega, um nome resolvido é suficiente para uma leitura. Portanto, os exemplos seguintes ainda devem ser aceitos:

```
int f() {
    int x;
    print(x);           // inicialização definida será verificada depois
}                       // retorno em todos os caminhos será verificado
    depois

int main() {
    int y = y;
    return 0;
}
```

Não implemente ainda estados de inicialização, junção entre ramos, efeito de laços, alcance de comandos ou análise de `fallthrough`. A Análise Semântica 2 acrescentará essas verificações sem alterar os vínculos e tipos desta etapa.

8 Execução, testes e restrições

Depois de copiar o lexer e o parser do grupo:

```
python -m pip install -r requirements-dev.txt
python runner.py test.mc
python -m pytest -q
```

Os testes públicos exercitam as interfaces, casos centrais e coordenadas. Os testes privados cobrem as mesmas regras com combinações adicionais e casos de borda; eles não introduzem exigências ausentes deste documento ou da especificação da MICROC.

Não é permitido alterar `symbols.py` ou `semantic_errors.py`, executar a análise semântica dentro do parser, alterar a AST para aceitar outra linguagem, procurar nomes no texto-fonte ou substituir o percurso estrutural por expressões regulares. A organização interna das passagens e seus auxiliares permanece escolha do grupo.

9 Checklist de entrega

Antes do prazo, verifique se:

- lexer e parser completos do grupo foram copiados para o repositório;
- todas as funções são conhecidas antes da resolução dos corpos;
- escopos, shadowing e espaços de nomes seguem o enunciado;
- toda expressão válida possui tipo e todo uso possui símbolo;
- diagnósticos acumulam categoria e coordenada corretas;
- nenhuma regra de fluxo da próxima etapa foi antecipada;
- `python -m pytest -q` passa; e
- o último commit aparece no repositório correto e no GitHub Actions.